



Spark & MLlib 1

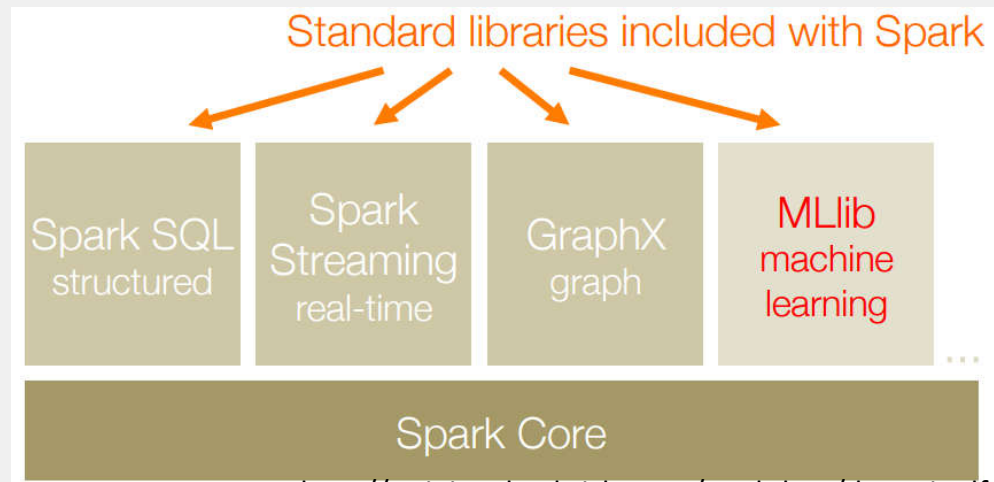
Podstawowe koncepcje oraz
rozproszone uczenie maszynowe

IWiUM

Bogdan Gliwa

Spark

- następca Hadoopa
- architektura



<http://training.databricks.com/workshop/datasci.pdf>

- aplikacje mogą być tworzone w Scali, Javie lub Pythonie

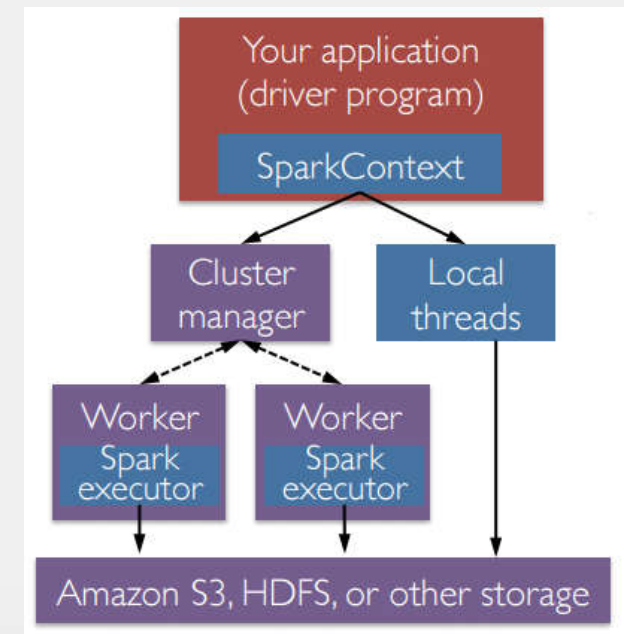
Podstawowe koncepcje platformy



- Spark Context
- dostęp do danych
 - RDD
 - DataFrames
 - <http://www.kdnuggets.com/2016/02/apache-spark-rdd-dataframe-dataset.html>
- transformations
- actions

SparkContext

- obiekt **SparkContext** przechowuje informacje jak i gdzie się podłączyć do klastra
- pySpark shell utworzony automatycznie pod zmienną **sc**
- SparkContext jest używany m.in. do tworzenia RDDs



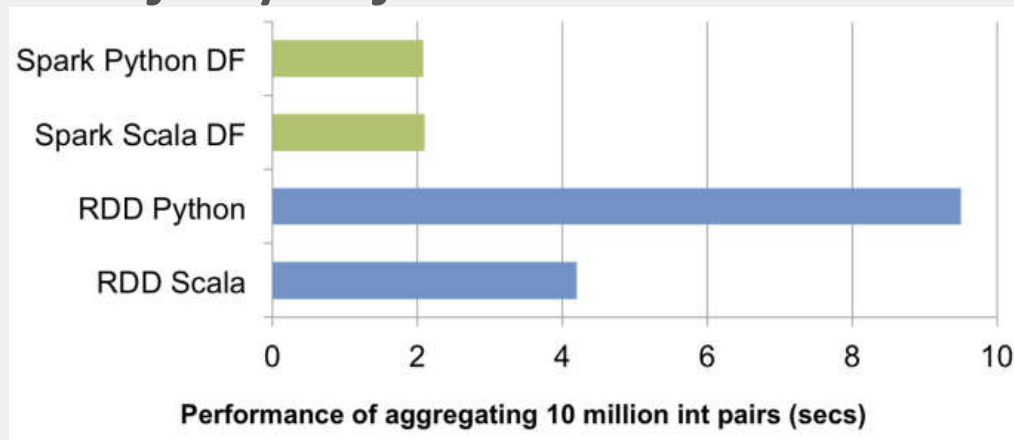
<https://courses.edx.org/c4x/BerkeleyX/CS100.1x/asset/Week2Lec4.pdf>

- RDD – Resilient Distributed Datasets
 - immutable once constructed
 - umożliwia wykonywanie operacji równoległe
 - tworzy się poprzez
 - zrównoleglenie kolekcji Pythona – operacja *parallelize()*
 - transformację istniejącego RDD
 - wczytanie pliku

```
>>> rdd = sc.parallelize([1, 2, 3])  
>>> rdd = sc.textFile('data.txt')
```

DataFrame

- rozproszona kolekcja danych zorganizowana w nazwane kolumny (typy kolumn są wnioskowane na podstawie wartości), jest schemat danych
- bardziej wydajne niż RDD



<https://courses.edx.org/c4x/BerkeleyX/CS100.1x/asset/Week3Lec5.pdf>

Transformations



- Transformations
 - lazy (not computed immediately)
- przykłady

Transformation	Description
<code>map(func)</code>	return a new distributed dataset formed by passing each element of the source through a function <i>func</i>
<code>filter(func)</code>	return a new dataset formed by selecting those elements of the source on which <i>func</i> returns true
<code>distinct([numTasks])</code>	return a new dataset that contains the distinct elements of the source dataset
<code>flatMap(func)</code>	similar to map, but each input item can be mapped to 0 or more output items (so <i>func</i> should return a Seq rather than a single item)

<https://courses.edx.org/c4x/BerkeleyX/CS100.1x/asset/Week2Lec4.pdf>

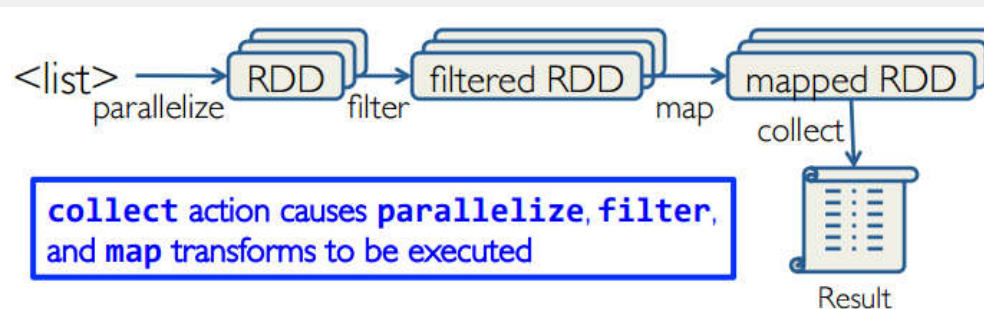
Actions

- powodują wykonanie algorytmów transformacji zbioru
- przykłady

Action	Description
<code>reduce(func)</code>	aggregate dataset's elements using function <i>func</i> . <i>func</i> takes two arguments and returns one, and is commutative and associative so that it can be computed correctly in parallel
<code>take(n)</code>	return an array with the first <i>n</i> elements
<code>collect()</code>	return all the elements as an array WARNING: make sure will fit in driver program
<code>takeOrdered(n, key=func)</code>	return <i>n</i> elements ordered in ascending order or as specified by the optional key function

<https://courses.edx.org/c4x/BerkeleyX/CS100.1x/asset/Week2Lec4.pdf>

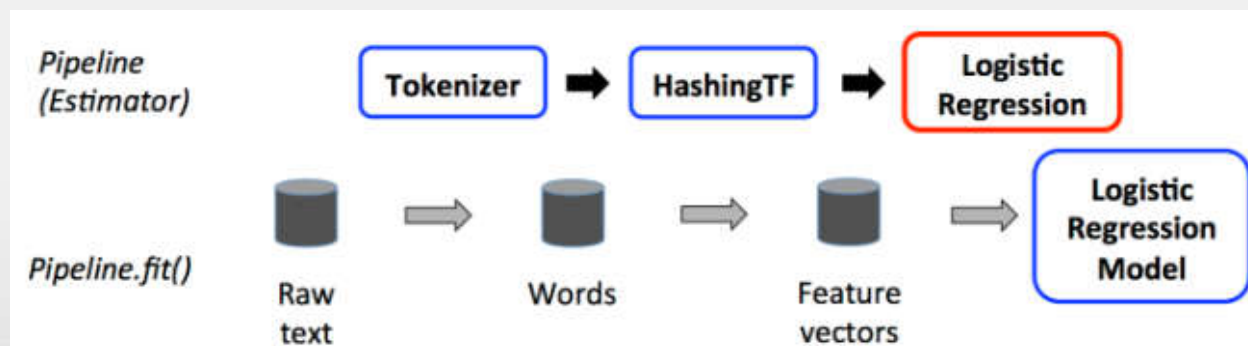
- przykładowy ciąg transformacji i akcji



- komponent Sparka
- zawiera algorytmy uczenia maszynowego
 - klasyfikacji: regresja logistyczna, NaiveBayes, Random Forests..
 - regresji: GLM
 - collaborative filtering: ALS
 - klasteryzacji: k-means
 - algebry liniowej: SVD, PCA
- zawiera 2 pakiety
 - **(py)spark.mllib** – starsze API, zbudowane na bazie **RDDs**
 - **(py)spark.ml** – nowsze API, zbudowane na bazie **DataFrames**, zalecane, wspiera pipelines
- <http://spark.apache.org/docs/latest/ml-guide.html>

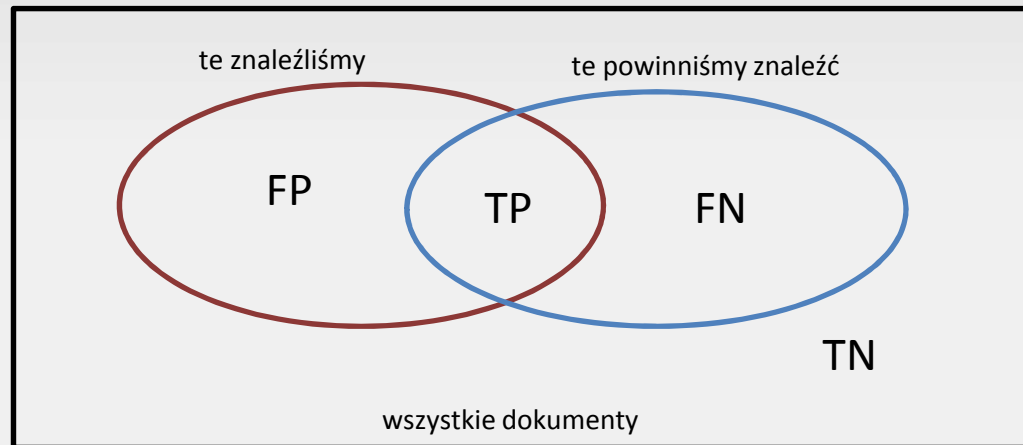
MLlib - model

- **DataFrame** – zbiór danych
- **Transformer** – algorytm przekształcający jeden zbiór danych na inny np. selekcja cech; ma metodę *transform()*
- **Estimator** – algorytm uczony na DataFrame, zwracający Transformer np. algorytm uczenia jest klasy Estimator, który jest trenowany na DataFrame i produkuje model; ma metodę *fit()*
- **Pipeline** – definiuje workflow poprzez łączenie obiektów typu Transformer i Estimator w ciąg



<http://spark.apache.org/docs/latest/ml-guide.html>

Klasyfikacja - metryki



$$\text{precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall})$$

$$\text{accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Krzywa ROC

- $TPR = TP/P$
 - recall
 - sensitivity
- $FPR = FP/N = FP/(FP+TN)$

